

32. 実測勾配推定ノード

solar_ws0129-main

2026-07-29

Contents

1 対象	3
2 このファイルを読む理由	3
3 呼び出し関係	3
3.1 どこから呼ばれるか	3
3.2 次に読むと理解が進むファイル	3
3.3 同一リポジトリ内の主要依存	3
4 ソース冒頭の把握	3
4.1 代表コード断片	3
5 主要構造の読み取り	4
5.1 主要クラス・関数	4
5.2 ファイルを上から読んだときの定義順	4
5.3 import 群の詳細	4
6 このファイルを読む前に必要な基礎知識	5
6.1 プログラム、プロセス、メモリ、オブジェクトを区別する	5
6.2 名前、代入、型、参照	5
6.3 関数、引数、戻り値、スコープ	6
6.4 module、package、import、console_scripts	6
6.5 例外、try/finally、with、資源解放	6
6.6 class、独自クラス、インスタンス、self、継承	6
6.7 先頭アンダースコア、__init__、名前付けの作法	7
6.8 rclpy、rcl、rmw、DDS/RTPS の層	7
6.9 ROS graph、Node、topic、service、action、parameter	8
6.10 callback、timer、Executor、spin、Callback Group	8
6.11 rqt_graph、ros2 CLI、rosviz をいつ使うか	8
6.12 制御、状態、入力、モデル予測制御 MPC	9
7 関数・クラスを上から順に解説	9
7.1 L10 クラス GradeNode	9

7.2	L11 関数 GradeNode.__init__	11
7.3	L43 関数 GradeNode._on_s_km	13
7.4	L46 関数 GradeNode._on_speed	13
7.5	L49 関数 GradeNode._on_altitude	14
7.6	L52 関数 GradeNode._on_gps	14
7.7	L56 関数 GradeNode._update_altitude	15
7.8	L63 関数 GradeNode._step	16
7.9	L89 関数 main	18
7.10	設定値と入力パラメータ	18
7.11	ROS topic I/O	19
8	処理の流れ	19
9	このファイルの位置づけ	19

1 対象

項目	内容
ファイル	mpc_solarcar/grade_node.py
ソース SHA-256	f67e3de2b8032378bb2cac9882ed85cd35629cc2dab61ee2f0ae17f7626d3131
種別	Python
区分	runtime node
役割	distance と altitude/GPS から grade を推定し /vehicle/grade へ publish する。
起動文脈	measure/live の監視・記録用の grade source。

2 このファイルを読む理由

distance と altitude/GPS から grade を推定し /vehicle/grade へ publish する。

- 現在の solar mpc_node は route profile 勾配を主に使い、/vehicle/grade を直接は使わない。
- ただし logger や補助用途では重要。

3 呼び出し関係

3.1 どこから呼ばれるか

- launch/solar_measurement.launch.py
- mpc_solarcar/live_launch.py

3.2 次に読むと理解が進むファイル

- 特記事項なし。

3.3 同一リポジトリ内の主要依存

- 同一リポジトリ内の明示 import は少ない。

4 ソース冒頭の把握

モジュール docstring または先頭意図の要約:

モジュール docstring は明示されていない。

4.1 代表コード断片

```
from sensor_msgs.msg import NavSatFix

class GradeNode(Node):
    def __init__(self):
        super().__init__('grade_node')
        self.declare_parameter('min_speed_kmh', 5.0)
        self.declare_parameter('altitude_alpha', 0.2)
        self.declare_parameter('min_delta_s_km', 0.01)
```

```

self.declare_parameter('gps_topic', '/sim/gps')
self.declare_parameter('altitude_topic', '/vehicle/altitude_m')

self.min_speed_kmh = float(self.get_parameter('min_speed_kmh').value)
self.alpha = float(self.get_parameter('altitude_alpha').value)
self.min_delta_s_km = float(self.get_parameter('min_delta_s_km').value)
self.gps_topic = self.get_parameter('gps_topic').value
self.altitude_topic = self.get_parameter('altitude_topic').value

self.altitude_m = math.nan
self.altitude_ema = math.nan
self.s_km = math.nan
self.speed_kmh = math.nan

```

5 主要構造の読み取り

主要クラスは GradeNode。主要関数は main。ROS パラメータ宣言は 5 件。ROS I/O は publisher 1 件、subscription 4 件。

5.1 主要クラス・関数

行	種別	名前	補足
10	class	GradeNode	bases: Node
11	function	__init__	args: self
43	function	_on_s_km	args: self, msg
46	function	_on_speed	args: self, msg
49	function	_on_altitude	args: self, msg
52	function	_on_gps	args: self, msg
56	function	_update_altitude	args: self, altitude_m
63	function	_step	args: self
89	function	main	

5.2 ファイルを上から読んだときの定義順

行	内容
10	クラス GradeNode を定義する。
89	関数 main を定義する。

5.3 import 群の詳細

行	import 文	このファイルで読む理由
1	import math	Python 標準のスカラー数値関数と定数を使うため。実際に参照する名前と行は使用位置欄で確認する。このファイル内での主な使用位置は L25, L26, L27, L28, L31, L53, L58, L64, ...。

2	<code>import time</code>	wall/monotonic time に基づく周期制御や freshness 判定を行うため。このファイル内での主な使用位置は L32。
4	<code>import rclpy</code>	ROS 2 Python ノードとして起動・spin するため。このファイル内での主な使用位置は L90, L92, L94。
5	<code>from rclpy.node import Node</code>	ROS 2 ノード本体の基底クラスとして使うため。このファイル内での主な使用位置は L10。
6	<code>from std_msgs. msg import Float32</code>	Float32、Bool、String などの標準 ROS message で値を publish または subscribe するため。このファイル内での主な使用位置は L34, L35, L36, L39, L43, L46, L49, L65, ...。
7	<code>from sensor_msgs. msg import NavSatFix</code>	GPS 緯度経度高度を ROS topic でやり取りするため。このファイル内での主な使用位置は L37, L52。

6 このファイルを読む前に必要な基礎知識

次の章は、構文や ROS 用語を既知と仮定しないための説明である。

6.1 プログラム、プロセス、メモリ、オブジェクトを区別する

ソースファイルはディスク上の文字列であり、それ自体は走っていない。Python 実行可能プログラムがソースを読み、OS がその実行を一つのプロセスとして管理する。プロセスには仮想メモリ、開いているファイル、スレッド、終了コードなどが対応する。

ソースファイル -> Python インタプリタが読み込む -> OS 上のプロセス -> Python オブジェクトをメモリに生成 -> 関数や callback を実行

「メモリ上に生成する」とは、実行中プロセスが使う記憶領域に、その値の型、属性、参照関係を表す実体を用意することである。変数は実体そのものというより、そのオブジェクトを指す名前として理解すると Python の代入が読みやすい。

```
node = MPCNode()
alias = node
# node と alias は同じオブジェクトを参照する。
```

一つのプロセスには複数スレッドを持たせられる。スレッドは同じプロセスのメモリを共有するため、受信 callback と最適化 callback が同じ self.z を書き換える場合は実行順と排他を検討する必要がある。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.2 名前、代入、型、参照

Python の代入文は、右辺を先に評価し、その結果のオブジェクトへ左辺の名前を結び付ける。‘x = f()’では、まず f を呼び、その戻り値が得られてから x が更新される。

‘float(...)’、‘int(...)’、‘bool(...)’は型名を呼び出して値を変換する。外部 CSV、YAML、ROS parameter から来た値は型が期待どおりとは限らないため、このリポジトリでは明示変換が多い。

‘None’は値が無いことを示す単一のオブジェクト、‘math.nan’は浮動小数点値ではあるが有効な数値ではないことを示す。両者は用途が異なるため、‘is None’と ‘math.isfinite’を使い分ける。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.3 関数、引数、戻り値、スコープ

‘def’は関数本体をその場で実行する文ではない。関数オブジェクトを作り、その名前を現在の名前空間へ登録する。‘f’は関数そのもの、‘f()’はその関数を呼んだ結果である。

仮引数は関数定義側の受け取り名、実引数は呼出側から渡す値である。位置引数、キーワード引数、既定値付き引数、‘*args’、‘**kwargs’は渡し方が異なる。

```
def clip_speed(value, minimum=0.0, maximum=110.0):  
    return min(maximum, max(minimum, value))  
  
v = clip_speed(120.0, maximum=90.0)
```

関数内で作った通常の名前はローカルスコープに属する。メソッドが‘self.z’を更新すればオブジェクトの状態が残るが、単に‘z’を更新しただけならその呼出しのローカル値である。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.4 module、package、import、console_scripts

Python ファイルは module として読み込める。複数 module をディレクトリにまとめたものが package である。‘from .model import SolarCarModel’の先頭の点は、現在と同じ package 内の model module を指す相対 import である。

import 時にはファイルのトップレベル文が上から一度実行される。‘def’や‘class’は関数・クラスを登録するが、その本体の通常処理は呼び出すまで走らない。

setuptools の‘console_scripts’は、端末で使う実行可能名と‘package.module:function’を対応付けるインストール時メタデータである。生成される実行用ラッパーは module を import し、指定関数を呼び、戻り値を終了コードとして扱う小さな入口である。

```
ROS launchのexecutable名 -> install済みconsole script -> mpc_solarcar.mpc_nodeをimport ->  
main()を呼び
```

根拠資料

- [setuptools 公式: Entry Points / Console Scripts](#)
- [Python 公式チュートリアル: Classes](#)

6.5 例外、try/finally、with、資源解放

例外は通常の戻り値とは別経路で異常を呼出元へ伝える。‘try/except’は想定した異常を処理し、‘finally’は成功・失敗にかかわらず後始末を行う。

‘with open(...) as f:’は context manager を使い、ブロックを出るとファイルを閉じる。CSV やログの破損を避けるため、開いた資源の所有者と閉じる場所を明確にする。

‘except Exception: pass’はノードを止めない利点がある一方、入力異常を隠して原因追跡を難しくする。安全に関係する値では、少なくとも頻度制限付き警告、異常カウンタ、fallback 状態の publish を検討する。

6.6 class、独自クラス、インスタンス、self、継承

クラスは、保持するデータと、そのデータを扱う関数を一つの型としてまとめる設計単位である。「独自クラス」とは Python や ROS 2 が最初から用意した型ではなく、このプロジェクトが目的に合わせて定義した新しい型を指す。

‘class MPCNode(Node)’は、rclpy の Node を基底クラスとして継承し、Node が持つ publisher、subscription、timer、parameter などの機能に MPC 固有機能を追加する。丸括弧内は関数引数ではなく基底クラス指定である。

‘MPCNode()’はクラスオブジェクトを呼び出して新しいインスタンスを作る。Python は新しい実体を用意し、その実体を第 1 引数 self として ‘__init__’へ渡す。‘self’は予約語ではなく慣習的な名前だが、この慣習を守ることでコードの意味が共有される。

```
class Counter:
    def __init__(self):
        self.value = 0

    def increment(self):
        self.value += 1

counter = Counter()
counter.increment()
```

‘super().__init__(...)’は継承元の初期化処理を呼ぶ。MPCNode ではこれを省くと ROS ノードとして必要な内部実体が作られず、‘create_publisher’などを正しく使えない。

根拠資料

- [Python 公式チュートリアル: Classes](#)
- [ROS 2 Humble 公式: Understanding nodes](#)

6.7 先頭アンダースコア、__init__、名前付けの作法

‘self.step_solar’の先頭 1 個のアンダースコアは「クラス内部で使う実装詳細」という慣習であり、アクセスを強制禁止する機構ではない。外部コードから呼べるが、公開 API として依存しない意思表示である。

‘__init__’の前後 2 個のアンダースコアは Python が定めた特殊メソッド名である。クラス生成時に自動で呼ばれる。任意の名前に前後 2 個を付けて独自仕様を作ることは避ける。

‘_’で始まるローカル変数は、未使用または外部へ見せない意図を示す場合がある。例えば ‘_base_csv’は戻り値として受け取る必要はあるが、その後の処理では使わないことを読者へ示す。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.8 rclpy、rcl、rmw、DDS/RTPS の層

rclpy は Python 利用者向け ROS 2 client library である。利用者の Node、publisher、subscriptionなどを Python API として提供し、その下で共通 C 層の rcl を利用する。

本プロジェクトのPythonコード -> rclpy -> rcl -> rmw -> DDS/RTPS実装 -> ネットワークまたは同一PC内通信

rcl は言語に依存しない共通 ROS 機能を提供する C API、rmw は ROS 2 と具体的 middleware 実装の境界である。DDS/RTPS 側が探索、serialize、publish/subscribe、request/replyなどを担う。executorの実行モデルは rcl だけで完結せず client library 側にも実装される。

根拠資料

- [ROS 2 Humble 公式: Internal ROS 2 interfaces](#)

6.9 ROS graph、Node、topic、service、action、parameter

ROS graph は、実行中 Node と、それらが持つ publisher、subscription、service、action などの接続関係である。Python クラス、プロセス、ROS graph 上の Node 名は関連するが同一物ではない。

topic は継続データ向けの非同期一方向 publish/subscribe、service は短い request/response、action は時間のかかる目標へ feedback、cancel、result を持たせる。parameter は Node ごとの設定値である。

publisher が送った時点と subscription callback が実行される時点は同じとは限らない。通信遅延、QoS queue、executor 待ちを挟むため、センサ値には timestamp と freshness 判定が必要になる。

根拠資料

- [ROS 2 Humble 公式: Understanding nodes](#)
- [ROS 2 Humble 公式: Topics, Services, Actions](#)
- [ROS 2 Humble 公式: Parameters](#)

6.10 callback、timer、Executor、spin、Callback Group

callback は、メッセージ受信、timer 満了、service request などのイベントが成立した後で Executor から呼ばれる関数である。登録時に 'self._on_speed' と括弧なしで渡すのは、今実行せず後で呼ぶ関数オブジェクトを渡すためである。

Executor は実行可能になった callback を見つけ、Callback Group の条件を確認して実行する。'spin()' は終了要求まで待機・dispatch を続けるため、通常運転中は main の次の行へ戻らない。

MutuallyExclusiveCallbackGroup は同じ group 内の callback を同時実行させない。別 group 間は MultiThreadedExecutor で並行実行し得る。group を分けただけでは共有属性への完全な排他にはならないため、複数 group が同じ状態を読む・書く場合は設計確認が必要である。

```
DDS受信またはtimer満了 -> wait setでready -> Executorが選択 -> Callback Groupが許可 -> worker threadがcallback実行 -> 終了後Executorへ戻る
```

根拠資料

- [ROS 2 公式: Executors](#)
- [ROS 2 Humble 公式: Internal ROS 2 interfaces](#)

6.11 rqt_graph、ros2 CLI、rosbag2 をいつ使うか

rqt_graph は接続関係を見る道具であり、数値の正しさや更新周期までは保証しない。起動直後、topic 名変更後、publisher が複数存在する疑いがある時に使う。

```
ros2 node list
ros2 node info /mpc_node
ros2 topic list -t
ros2 topic info -v /planner/speed_cmd
ros2 topic hz /planner/speed_cmd
ros2 topic echo /planner/status
rqt_graph
```

rosbag2 は topic メッセージを時系列のまま記録・再生する。通信不具合、freshness、再計画 trigger、実車と SILS の差を再現可能にするため、本番前試験では制御入力だけでなく原因となる全 telemetry、status、parameter 情報を記録する。


```

ros2 bag record -o outputs/bags/preflight \
  /vehicle/s_km /vehicle/speed_kmh /vehicle/batt_soc \
  /vehicle/batt_temp_c /vehicle/batt_current_a /vehicle/batt_voltage_v \
  /planner/upper_speed_cmd /planner/speed_cmd /planner/status

ros2 bag info outputs/bags/preflight
ros2 bag play outputs/bags/preflight --clock

```

bag 再生時は QoS 互換性、simulation time、外部 publisher との二重入力に注意する。実車 Node を同時に動かす場合は namespace または remapping で入力源を明確に分離する。

根拠資料

- [ROS 2 Humble 公式: rqt_graph](#)
- [ROS 2 Humble 公式: Recording and playing back data](#)
- [ROS 2 Humble 公式: Understanding nodes](#)

6.12 制御、状態、入力、モデル予測制御 MPC

制御対象の内部を表す状態を x 、操作入力を u 、外乱・予報を w とすると、離散モデルは ' $x[k+1] = f(x[k], u[k], w[k])$ ' と書ける。ソーラーカーでは SoC、電池温度、距離、速度などが状態候補、速度目標や駆動トルクが入力候補になる。

$$\min_{u_0, \dots, u_{N-1}} \sum_{k=0}^{N-1} \ell(x_k, u_k, w_k) + V_f(x_N)$$

MPC は現在状態から N ステップ先まで予測し、目的関数と制約を満たす入力系列を求める。ただし実際に適用するのは通常先頭入力だけで、次回は新しい実測状態から再び解く。これが **receding horizon** である。

予測モデル、目的関数、制約、ホライズン、solver、初期値のどれかが変わると答えも変わる。「MPC を使う」だけでは仕様は決まらず、これらを単位付きで追う必要がある。

根拠資料

- [SciPy 公式: scipy.optimize.minimize](#)

7 関数・クラスを上から順に解説

7.1 L10 クラス GradeNode

- 行範囲: L10–L86
- 所属: module 直下
- 定義: `GradeNode(bases=Node)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: 特に明示されていない。
- 戻り値の要点: 戻り値が無いが、`return` 文が明示されていない。
- 主なローカル代入名: 特になし。

- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 0

この定義を読むための Python 構文

- `class` 文は新しい型を定義する。丸括弧内は継承する基底クラスである。

上から順の処理

1. 関数 `__init__` を定義する。
2. 関数 `_on_s_km` を定義する。
3. 関数 `_on_speed` を定義する。
4. 関数 `_on_altitude` を定義する。
5. 関数 `_on_gps` を定義する。
6. 関数 `_update_altitude` を定義する。
7. 関数 `_step` を定義する。

代表コード断片

```
class GradeNode(Node):
    def __init__(self):
        super().__init__('grade_node')
        self.declare_parameter('min_speed_kmh', 5.0)
        self.declare_parameter('altitude_alpha', 0.2)
        self.declare_parameter('min_delta_s_km', 0.01)
        self.declare_parameter('gps_topic', '/sim/gps')
        self.declare_parameter('altitude_topic', '/vehicle/altitude_m')

        self.min_speed_kmh = float(self.get_parameter('min_speed_kmh').value)
        self.alpha = float(self.get_parameter('altitude_alpha').value)
        self.min_delta_s_km = float(self.get_parameter('min_delta_s_km').value)
        self.gps_topic = self.get_parameter('gps_topic').value
        self.altitude_topic = self.get_parameter('altitude_topic').value

        self.altitude_m = math.nan
        self.altitude_ema = math.nan
        self.s_km = math.nan
        self.speed_kmh = math.nan
        self.last_s_km = None
        self.last_alt_ema = None
        self.grade = math.nan
        self.last_update = time.monotonic()

        self.create_subscription(Float32, '/vehicle/s_km', self._on_s_km, 10)
        self.create_subscription(Float32, '/vehicle/speed_kmh', self._on_speed, 10)
        self.create_subscription(Float32, self.altitude_topic, self._on_altitude, 10)
        self.create_subscription(NavSatFix, self.gps_topic, self._on_gps, 10)

        self.pub_grade = self.create_publisher(Float32, '/vehicle/grade', 10)
        self.timer = self.create_timer(1.0, self._step)
        self.get_logger().info('GradeNode started.')
```

```
def _on_s_km(self, msg: Float32):
    self.s_km = float(msg.data)
...
```

7.2 L11 関数 GradeNode.__init__

- 行範囲: L11–L41
- 所属: GradeNode
- 定義: `__init__(self)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `__init__`, `create_publisher`, `create_subscription`, `create_timer`, `declare_parameter`, `float`, `get_logger`, `get_parameter`, `info`, `monotonic`, `super`
- 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: `self._on_altitude`, `self._on_gps`, `self._on_s_km`, `self._on_speed`, `self._step`, `self.altitude_topic`, `self.create_publisher`, `self.create_subscription`, `self.create_timer`, `self.declare_parameter`, `self.get_logger`, `self.get_parameter`, `self.gps_topic`
- 更新する主なインスタンス属性: `self.alpha`, `self.altitude_ema`, `self.altitude_m`, `self.altitude_topic`, `self.gps_topic`, `self.grade`, `self.last_alt_ema`, `self.last_s_km`, `self.last_update`, `self.min_delta_s_km`, `self.min_speed_kmh`, `self.pub_grade`, `self.s_km`, `self.speed_kmh`, `self.timer`
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。

上から順の処理

1. `super().__init__(...)` を実行する。
2. `self.declare_parameter(...)` を実行する。
3. `self.declare_parameter(...)` を実行する。
4. `self.declare_parameter(...)` を実行する。
5. `self.declare_parameter(...)` を実行する。
6. `self.declare_parameter(...)` を実行する。
7. `self.min_speed_kmh` に `float(self.get_parameter('min_speed_kmh').value)` の結果を代入する。
8. `self.alpha` に `float(self.get_parameter('altitude_alpha').value)` の結果を代入する。
9. `self.min_delta_s_km` に `float(self.get_parameter('min_delta_s_km').value)` の結果を代入する。
10. `self.gps_topic` に `self.get_parameter('gps_topic').value` の結果を代入する。
11. `self.altitude_topic` に `self.get_parameter('altitude_topic').value` の結果を代入する。

12. self.altitude_m に math.nan の結果を代入する。
13. self.altitude_ema に math.nan の結果を代入する。
14. self.s_km に math.nan の結果を代入する。
15. self.speed_kmh に math.nan の結果を代入する。
16. self.last_s_km に None の結果を代入する。
17. self.last_alt_ema に None の結果を代入する。
18. self.grade に math.nan の結果を代入する。
19. self.last_update に time.monotonic() の結果を代入する。
20. self.create_subscription(...) を実行する。
21. self.create_subscription(...) を実行する。
22. self.create_subscription(...) を実行する。
23. self.create_subscription(...) を実行する。
24. self.pub_grade に self.create_publisher(Float32, '/vehicle/grade', 10) の結果を代入する。
25. self.timer に self.create_timer(1.0, self._step) の結果を代入する。
26. self.get_logger().info(...) を実行する。

代表コード断片

```
def __init__(self):
    super().__init__('grade_node')
    self.declare_parameter('min_speed_kmh', 5.0)
    self.declare_parameter('altitude_alpha', 0.2)
    self.declare_parameter('min_delta_s_km', 0.01)
    self.declare_parameter('gps_topic', '/sim/gps')
    self.declare_parameter('altitude_topic', '/vehicle/altitude_m')

    self.min_speed_kmh = float(self.get_parameter('min_speed_kmh').value)
    self.alpha = float(self.get_parameter('altitude_alpha').value)
    self.min_delta_s_km = float(self.get_parameter('min_delta_s_km').value)
    self.gps_topic = self.get_parameter('gps_topic').value
    self.altitude_topic = self.get_parameter('altitude_topic').value

    self.altitude_m = math.nan
    self.altitude_ema = math.nan
    self.s_km = math.nan
    self.speed_kmh = math.nan
    self.last_s_km = None
    self.last_alt_ema = None
    self.grade = math.nan
    self.last_update = time.monotonic()

    self.create_subscription(Float32, '/vehicle/s_km', self._on_s_km, 10)
    self.create_subscription(Float32, '/vehicle/speed_kmh', self._on_speed, 10)
    self.create_subscription(Float32, self.altitude_topic, self._on_altitude, 10)
    self.create_subscription(NavSatFix, self.gps_topic, self._on_gps, 10)

    self.pub_grade = self.create_publisher(Float32, '/vehicle/grade', 10)
    self.timer = self.create_timer(1.0, self._step)
    self.get_logger().info('GradeNode started.')
```

7.3 L43 関数 `GradeNode._on_s_km`

- 行範囲: L43–L44
- 所属: `GradeNode`
- 定義: `_on_s_km(self, msg: Float32)`
- `docstring`: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `float`
- 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: `self.s_km`
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。

上から順の処理

1. `self.s_km` に `float(msg.data)` の結果を代入する。

代表コード断片

```
def _on_s_km(self, msg: Float32):  
    self.s_km = float(msg.data)
```

7.4 L46 関数 `GradeNode._on_speed`

- 行範囲: L46–L47
- 所属: `GradeNode`
- 定義: `_on_speed(self, msg: Float32)`
- `docstring`: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `float`
- 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: `self.speed_kmh`
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。

上から順の処理

1. `self.speed_kmh` に `float(msg.data)` の結果を代入する。

代表コード断片

```
def _on_speed(self, msg: Float32):  
    self.speed_kmh = float(msg.data)
```

7.5 L49 関数 `GradeNode._on_altitude`

- 行範囲: L49–L50
- 所属: `GradeNode`
- 定義: `_on_altitude(self, msg: Float32)`
- `docstring`: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `_update_altitude`, `float`
- 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: `self._update_altitude`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。

上から順の処理

1. `self._update_altitude(...)` を実行する。

代表コード断片

```
def _on_altitude(self, msg: Float32):  
    self._update_altitude(float(msg.data))
```

7.6 L52 関数 `GradeNode._on_gps`

- 行範囲: L52–L54
- 所属: `GradeNode`
- 定義: `_on_gps(self, msg: NavSatFix)`
- `docstring`: 明示されていない。

- このブロックが直接呼ぶ主な関数・メソッド: `_update_altitude`, `float`, `isfinite`
- 戻り値の要点: 戻り値が無いが、`return` 文が明示されていない。
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: `self._update_altitude`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 1、ループ 0、`try` 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。

上から順の処理

1. 条件 `math.isfinite(msg.altitude)` を判定し、真なら内部処理を行う。
2. `self._update_altitude(...)` を実行する。

代表コード断片

```
def _on_gps(self, msg: NavSatFix):
    if math.isfinite(msg.altitude):
        self._update_altitude(float(msg.altitude))
```

7.7 L56 関数 `GradeNode._update_altitude`

- 行範囲: L56–L61
- 所属: `GradeNode`
- 定義: `_update_altitude(self, altitude_m: float)`
- `docstring`: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `isfinite`
- 戻り値の要点: 戻り値が無いが、`return` 文が明示されていない。
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: `self.alpha`, `self.altitude_ema`
- 更新する主なインスタンス属性: `self.altitude_ema`, `self.altitude_m`
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 1、ループ 0、`try` 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。

上から順の処理

1. self.altitude_m に altitude_m の結果を代入する。
2. 条件 not math.isfinite(self.altitude_ema) を判定し、真なら内部処理を行う。
3. self.altitude_ema に altitude_m の結果を代入する。
4. 上の条件が偽の場合:
5. self.altitude_ema に self.alpha * altitude_m + (1.0 - self.alpha) * self.altitude_ema の結果を代入する。

代表コード断片

```
def _update_altitude(self, altitude_m: float):
    self.altitude_m = altitude_m
    if not math.isfinite(self.altitude_ema):
        self.altitude_ema = altitude_m
    else:
        self.altitude_ema = self.alpha * altitude_m + (1.0 - self.alpha) * self.altitude_ema
```

7.8 L63 関数 GradeNode._step

- 行範囲: L63–L86
- 所属: GradeNode
- 定義: _step(self)
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: Float32, float, isfinite, publish
- 戻り値の要点: 戻り値が無い、return 文が明示されていない。
- 主なローカル代入名: delta_alt, delta_s_km, grade
- 読み取る主なインスタンス属性: self.altitude_ema, self.grade, self.last_alt_ema, self.last_s_km, self.min_delta_s_km, self.min_speed_kmh, self.pub_grade, self.s_km, self.speed_kmh
- 更新する主なインスタンス属性: self.grade, self.last_alt_ema, self.last_s_km
- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 4、ループ 0、try 0

この定義を読むための Python 構文

- 第 1 引数 self は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。

上から順の処理

1. 条件 not math.isfinite(self.s_km) or not math.isfinite(self.altitude_ema) を判定し、真なら内部処理を行う。
2. self.pub_grade.publish(...) を実行する。
3. を返す。
4. 条件 not math.isfinite(self.speed_kmh) or self.speed_kmh < self.min_speed_kmh を判定し、真なら内部処理を行う。

5. self.pub_grade.publish(...) を実行する。
6. を返す。
7. 条件 self.last_s_km is None or self.last_alt_ema is None を判定し、真なら内部処理を行う。
8. self.last_s_km に self.s_km の結果を代入する。
9. self.last_alt_ema に self.altitude_ema の結果を代入する。
10. self.pub_grade.publish(...) を実行する。
11. を返す。
12. delta_s_km に self.s_km - self.last_s_km の結果を代入する。
13. 条件 delta_s_km < self.min_delta_s_km を判定し、真なら内部処理を行う。
14. self.pub_grade.publish(...) を実行する。
15. を返す。
16. delta_alt に self.altitude_ema - self.last_alt_ema の結果を代入する。
17. grade に delta_alt / (delta_s_km * 1000.0) の結果を代入する。
18. self.grade に float(grade) の結果を代入する。
19. self.last_s_km に self.s_km の結果を代入する。
20. self.last_alt_ema に self.altitude_ema の結果を代入する。
21. self.pub_grade.publish(...) を実行する。

代表コード断片

```
def _step(self):
    if not math.isfinite(self.s_km) or not math.isfinite(self.altitude_ema):
        self.pub_grade.publish(Float32(data=math.nan))
        return
    if not math.isfinite(self.speed_kmh) or self.speed_kmh < self.min_speed_kmh:
        self.pub_grade.publish(Float32(data=self.grade if math.isfinite(self.grade) else
math.nan))
        return
    if self.last_s_km is None or self.last_alt_ema is None:
        self.last_s_km = self.s_km
        self.last_alt_ema = self.altitude_ema
        self.pub_grade.publish(Float32(data=math.nan))
        return

    delta_s_km = self.s_km - self.last_s_km
    if delta_s_km < self.min_delta_s_km:
        self.pub_grade.publish(Float32(data=self.grade if math.isfinite(self.grade) else
math.nan))
        return

    delta_alt = self.altitude_ema - self.last_alt_ema
    grade = delta_alt / (delta_s_km * 1000.0)
    self.grade = float(grade)
    self.last_s_km = self.s_km
    self.last_alt_ema = self.altitude_ema
    self.pub_grade.publish(Float32(data=self.grade))
```

7.9 L89 関数 main

- 行範囲: L89–L94
- 所属: module 直下
- 定義: `main()`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `GradeNode`, `destroy_node`, `init`, `shutdown`, `spin`
- 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。
- 主なローカル代入名: `node`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 0

この定義を読むための Python 構文

- 特別な構文上の補足は少ない。

上から順の処理

1. `rclpy.init(...)` を実行する。
2. `node` に `GradeNode()` の結果を代入する。
3. `rclpy.spin(...)` を実行する。
4. `node.destroy_node(...)` を実行する。
5. `rclpy.shutdown(...)` を実行する。

代表コード断片

```
def main():  
    rclpy.init()  
    node = GradeNode()  
    rclpy.spin(node)  
    node.destroy_node()  
    rclpy.shutdown()
```

7.10 設定値と入力パラメータ

行	名前	既定値・補足
13	<code>min_speed_kmh</code>	5.0
14	<code>altitude_alpha</code>	0.2
15	<code>min_delta_s_km</code>	0.01
16	<code>gps_topic</code>	/sim/gps
17	<code>altitude_topic</code>	/vehicle/altitude_m

7.11 ROS topic I/O

行	種別	topic	callback / 補足
39	publisher	/vehicle/grade	publisher
34	subscription	/vehicle/s_km	self._on_s_km
35	subscription	/vehicle/speed_kmh	self._on_speed
36	subscription	self.altitude_topic	self._on_altitude
37	subscription	self.gps_topic	self._on_gps

8 処理の流れ

1. 初期化時に設定値や入力パスを読み込む。
2. publisher / subscription / timer を準備する。
3. timer callback 周期で主処理を進める。

9 このファイルの位置づけ

この資料群では、`mpc_solarcar/grade_node.py` を **runtime node** の中核として扱う。したがって、ここで扱う処理は単独で完結せず、前段の `profile / launch / telemetry` と後段の `planner / logger / report` へ繋がる。